

FinTrack

Backend Complete Technical & Code Audit

Exact uploaded ZIP • easy language • professional technical depth

AUDIT SCOPE

I inspected the uploaded source project: **78 Java files, 1 test file**, Maven configuration and application configuration. Generated Git/build artifacts were excluded from the application analysis.

This is the point where we stop treating code as copy-paste. The goal is to understand the mechanics underneath every major feature so you can explain, debug and extend the backend yourself.

1. Architecture — the whole backend in one picture

FinTrack follows a layered architecture. The most important thing is not memorizing folders; it is understanding responsibility. A request crosses several layers, and each layer answers a different question.

```
HTTP request
↓
Spring Security Filter Chain
↓
Controller ← HTTP concerns
↓
DTO + Bean Validation ← API contract
↓
Service ← business rules / orchestration
↓
Repository / Specification / @Query ← persistence
↓
JPA + Hibernate ← ORM
↓
MySQL ← actual data storage
↑
Entity / projection
↑
Service mapping + calculations
↑
Response DTO
↑
JSON HTTP response
```

Layer	Responsibility	Examples
Controller	Receives HTTP requests and returns HTTP responses	ExpenseController, BudgetController
DTO	Input/output contract	CreateExpenseRequest, ExpenseResponse
Service	Business logic and use cases	ExpenseService, BudgetService
Repository	Database access	ExpenseRepository, IncomeRepository
Entity	Persistent object mapping	Expense, Budget, Income
Specification	Dynamic query predicates	ExpenseSpecification
Security	Identity and request authentication	JwtService, JwtAuthenticationFilter
Exception layer	Converts failures to API errors	GlobalExceptionHandler

Professional rule: controllers should not contain complex business logic; repositories should not decide business policy; entities should represent persistent/domain state; services orchestrate use cases.

2. Java mechanics you have been using without seeing them

Concept	FinTrack example	What you must understand
Class	Expense, User, ExpenseService	Fields + methods; blueprint for objects.
Object	new Expense()	Runtime instance of a class.
Interface	UserRepository	Contract; Spring Data provides implementation.
Generics	JpaRepository	Type parameters: entity type + ID type.
private/final	private final ExpenseRepository	Encapsulation + stable dependency reference.
Optional	findByEmail()	Result may be absent.
BigDecimal	amount, totals, percentages	Money-safe decimal arithmetic.
LocalDate	expenseDate	Date without time-of-day.
LocalDateTime	createdAt	Date + time.
Lambda	stream map / sort	Function-like expression used by Java functional APIs.
List / Map	dashboard aggregation	Collection and key/value data structures.
RuntimeException	custom exceptions	Failure path that can be handled centrally.

2.1 Constructor injection

```
@Service
public class ExpenseService {
    private final ExpenseRepository expenseRepository;

    public ExpenseService(ExpenseRepository expenseRepository) {
        this.expenseRepository = expenseRepository;
    }
}
```

Spring creates the ExpenseService bean. It sees that the constructor requires ExpenseRepository and supplies that dependency. This is Dependency Injection (DI). The larger principle is Inversion of Control (IoC): object creation/wiring is managed by the framework instead of every class constructing its dependencies itself.

INTERVIEW VERSION

“Constructor injection makes required dependencies explicit, supports final fields and makes the class easier to unit test because dependencies can be supplied from the test.”

2.2 BigDecimal

```
BigDecimal savings =
    totalIncome.subtract(totalExpense);

BigDecimal rate =
    savings.divide(totalIncome, 4, RoundingMode.HALF_UP)
        .multiply(BigDecimal.valueOf(100));
```

Do not replace this with double for financial calculations. BigDecimal gives decimal arithmetic plus explicit scale and rounding. Database DECIMAL precision/scale is a separate persistence concern.

3. Spring Boot + REST — how a request becomes Java code

Annotation	Technical meaning
@RestController	Registers a controller whose return values are written to the response body.
@RequestMapping	Common URL prefix.
@PostMapping / @GetMapping / @PutMapping / @DeleteMapping	Map HTTP methods/paths to Java methods.
@RequestBody	Jackson converts JSON into a Java object.
@PathVariable	Reads values such as {expenseId} from the URL.
@RequestParam	Reads query parameters such as page, size and filters.
@Valid	Runs Bean Validation on the request DTO.
ResponseEntity	Controls HTTP status/body explicitly.

Example: create expense

```
POST /api/v1/expenses
↓
JSON
↓ Jackson deserialization
CreateExpenseRequest
↓ @Valid
Bean Validation
↓
ExpenseController
↓
ExpenseService
↓
ExpenseRepository
↓
MySQL
```

Serialization is Java object → JSON. Deserialization is JSON → Java object. Jackson, integrated through Spring Web, performs this conversion.

DTO PRINCIPLE

A DTO is an API contract. It prevents your persistence model from automatically becoming your public API and lets request/response shapes evolve independently.

Validation has two levels

Bean Validation: @NotBlank, @NotNull, @DecimalMin, @Email, @Size. **Business validation:** future dates, budget overlap, goal contribution limits, amount ranges. The first checks input shape; the second enforces domain rules.

4. Maven + configuration — what runs your backend

Dependency/config	Purpose
spring-boot-starter-web	Spring MVC/REST + JSON/web infrastructure.
spring-boot-starter-data-jpa	Spring Data JPA + Hibernate integration.
spring-boot-starter-security	Security filter chain/authentication.
spring-boot-starter-validation	Jakarta Bean Validation.
mysql-connector-j	MySQL JDBC driver.
jjwt-api/impl/jackson	JWT creation and parsing.
Actuator	Monitoring/management foundation.
starter-test + security-test	Automated testing support.
Java 21	Declared Java target.
<pre>spring.datasource.url = jdbc:mysql://localhost:3306/fintrack spring.jpa.hibernate.ddl-auto = update spring.jpa.show-sql = true server.port = 8080 jwt.expiration = 86400000</pre>	

The source also contains a hardcoded DB username/password and a hardcoded JWT signing secret. These values must not be committed as real production secrets. Use environment variables/secret management and rotate any real secret that has already been exposed.

HIGH PRIORITY SECURITY FINDING

If the JWT secret or database password in the uploaded file is real and has ever reached a remote/public Git repository, hiding it now is not enough. Rotate it and replace it with externally supplied configuration.

ddl-auto=update: convenient for development, but production systems should use controlled schema migrations (for example Flyway/Liquibase). **show-sql=true:** useful for learning SQL, but production logging needs deliberate configuration.

5. Authentication — registration and login

```
REGISTER
AuthController
↓
AuthService
↓ existsByEmail()
↓ BCrypt.encode(password)
↓ User entity
↓ UserRepository.save()
↓ database

LOGIN
AuthController
↓
AuthService
↓ findByEmail()
↓ BCrypt.matches(raw, storedHash)
↓ JwtService.generateToken(email)
↓ LoginResponse(token)
```

BCrypt is hashing, not encryption. The original password is not recovered from the stored hash. `matches()` verifies the supplied password against the stored hash.

Class	Technical responsibility
RegisterRequest	Validated registration input.
LoginRequest	Login input; currently lacks validation annotations.
User	JPA entity for account data.
UserRepository	<code>existsByEmail</code> + <code>findByEmail</code> .
AuthService	Credential logic, hashing and token creation.
AuthController	HTTP boundary for <code>/register</code> and <code>/login</code> .
LoginResponse	Returns message + JWT token.

5.1 Exact source gaps in authentication

`RegisterRequest` has `@Size` on password but not `@NotBlank`, so null is not rejected by that constraint. `LoginRequest` has no `@NotBlank`/`@Email` validation. `AuthService` returns HTTP 200 from the controller even when it creates a “User not found” or “Invalid Password” `LoginResponse`. This also exposes account-existence information.

PRODUCTION LOGIN RULE

Use one generic authentication failure response and normally return 401 Unauthorized. Do not reveal whether an email exists. Normalize emails consistently before lookup/storage.

6. JWT + SecurityFilterChain — the most important security mechanics

JWT

- Header → algorithm/type
- Payload → claims such as subject + expiry
- Signature → integrity/authenticity

```
generateToken(email)
↓
setSubject(email)
setIssuedAt(...)
setExpiration(...)
signWith(HS256 key)
compact()
```

A normal JWT is encoded, not encrypted. Its payload should therefore never contain passwords, UPI PINs or secrets.

```
Authorization: Bearer <JWT>
↓
JwtAuthenticationFilter
↓
read header
↓
strip "Bearer "
↓
JwtService.isValidToken()
↓
extractEmail()
↓
UsernamePasswordAuthenticationToken
↓
SecurityContextHolder
↓
Controller/Service sees Authentication
```

The filter is a `OncePerRequestFilter`, so its `doFilterInternal` logic runs once for each request. If the token is absent/invalid, it currently continues the filter chain and lets Spring Security ultimately reject protected access. If valid, it establishes Authentication for that request.

SecurityConfig	Meaning
CSRF disabled	Chosen for the current stateless JWT REST architecture; not a universal rule for all web apps.
STATELESS	No HTTP session is used as the authentication mechanism.
permitAll register/login	Public authentication endpoints.
anyRequest authenticated	All other routes require authentication.
addFilterBefore	JWT filter is inserted before UsernamePasswordAuthenticationFilter.

CURRENT LIMIT

Authentication currently uses `Collections.emptyList()` for authorities. Identity/ownership works, but the `User.role` field is not yet translated into Spring Security authorities. Full USER/ADMIN authorization is a future hardening step.

7. Exception handling — Java failure → HTTP response

```
Service:
throw new ExpenseNotFoundException("Expense not found");

GlobalExceptionHandler:
@ExceptionHandler(ExpenseNotFoundException.class)
↓
HTTP 404
ApiResponse(404, message)
```

Exception	Intended status
EmailAlreadyExistsException	409 Conflict
ExpenseNotFoundException	404 Not Found
BudgetNotFoundException	404 Not Found
IncomeNotFoundException	404 Not Found
FinancialGoalNotFoundException	404 Not Found
InvalidBudgetException	400 Bad Request
InvalidIncomeException	400 Bad Request
InvalidFinancialGoalException	400 Bad Request

The uploaded GlobalExceptionHandler does not handle InvalidExpenseException or CategoryNotFoundException, even though services can throw them. It also defines ValidationErrorResponse but does not currently show a handler for MethodArgumentNotValidException. Therefore the intended uniform error contract is incomplete.

ARCHITECTURE RULE

The Service should decide **what business rule failed**; the exception layer should decide **how that failure is represented over HTTP**.

8. Expense — the core reusable CRUD/search pattern

Class	Role
ExpenseController	POST/GET/GET-by-id/PUT/DELETE + query params.
CreateExpenseRequest	Validated input.
ExpenseResponse	Output DTO including category information.
ExpensePageResponse	Pagination wrapper.
Expense	JPA entity.
Category	JPA category entity.
ExpenseRepository	CRUD + SUM/GROUP BY queries.
ExpenseSpecification	Dynamic predicates.
ExpenseService	Business rules + mapping.

8.1 Ownership is security

```
Optional<Expense> findByIdAndUserId(  
    Long id,  
    Long userId  
);
```

This is not merely a convenience query. In a multi-user finance application, a resource ID does not prove ownership. The query requires both the resource ID and the authenticated user's ID. The same pattern appears for Budget, Income and FinancialGoal.

8.2 Specifications

```
predicates:  
    user.id = authenticatedUserId  
    + optional categoryId  
    + optional startDate  
    + optional endDate  
    + optional minAmount  
    + optional maxAmount  
    + optional title/description search  
  
criteriaBuilder.and(allPredicates)
```

JpaSpecificationExecutor lets a repository execute these dynamically constructed predicates. This avoids creating a separate repository method for every filter combination.

8.3 Pagination

```
PageRequest.of(  
    page,  
    size,  
    Sort.by(direction, safeSortBy)  
)  
  
repository.findAll(specification, pageable)
```

Page contains content plus metadata. Pageable represents requested page/size/sort. Your services also restrict page size to 1–100, which protects the API from obviously huge page requests.

9. Budget — business rules beyond CRUD

Budget validates date order, category existence and overlapping periods. It also calculates spending by combining Budget data with Expense data.

```
existing.startDate <= newEndDate  
AND existing.endDate >= newStartDate
```

→ overlapping periods

The update query excludes the current budget ID so a budget does not conflict with itself. The same user/category restriction is included in the overlap query.

BudgetResponse	How calculated
spentAmount	SUM expenses for same user/category inside budget dates.
remainingAmount	budget amount – spent.
percentageUsed	spent ÷ budget × 100.
UPCOMING	Today before start date.
COMPLETED	Today after end date.
EXCEEDED	Spending at/above budget in normal response mapping.
ON_TRACK	Otherwise active and below budget.

Exact source finding: BudgetService.calculateStatus() uses >= for EXCEEDED, while getAllBudgetsForDashboard() uses >. Exactly 100% utilization can therefore receive different statuses. Centralize this calculation to one method.

Performance finding: mapping each budget calls calculateTotalSpent separately. With many budgets this can become many aggregate queries. Later we should consider batch aggregation.

10. Income — aggregation and reusable filtering

```
SELECT YEAR(i.incomeDate),
       MONTH(i.incomeDate),
       COALESCE(SUM(i.amount), 0)
FROM Income i
WHERE i.user.id = :userId
GROUP BY YEAR(i.incomeDate), MONTH(i.incomeDate)
ORDER BY YEAR(i.incomeDate), MONTH(i.incomeDate)
```

This query returns aggregate rows, not Income entities. The service therefore uses List. Position 0 = year, 1 = month, 2 = amount. This is a valid projection pattern but not strongly type-safe.

FUTURE IMPROVEMENT

Use typed projections or DTO/record queries for aggregate results. It makes the contract explicit and reduces runtime casting errors.

IncomeSpecification applies user ownership plus optional income type, date range, amount range and source/description search. This is the same Specification pattern you should recognize from Expense.

11. Financial goals — state, history and invariants

Rule	Source behavior
Target amount > 0	Bean Validation.
Target date not in past	FinancialGoalService.
Update target >= current amount	FinancialGoalService.
Contribution date not future	FinancialGoalService.
Contribution cannot exceed target	newCurrentAmount check.
Progress	$\text{current} \div \text{target} \times 100$, capped at 100.
Status	COMPLETED / OVERDUE / IN_PROGRESS.

GoalContribution is a separate entity because currentAmount is current state while contributions are history. This lets the product answer both “what is the current progress?” and “what transactions created that progress?”.

```
contributionRepository.save(contribution);

goal.setCurrentAmount(newCurrentAmount);
goalRepository.save(goal);
```

These two writes represent one business event. The uploaded service does not show @Transactional around addContribution. This is a key next-level concept: transaction boundaries protect atomicity, while concurrency strategy may additionally be required when two contributions happen at the same time.

RACE CONDITION EXAMPLE

Target ■10,000; current ■9,000. Two requests each contribute ■1,000. Both can read ■9,000 before either writes. A simple if-check is not enough to guarantee correctness under concurrency.

12. Dashboard — cross-module orchestration

Endpoint	Source logic
summary	Income totals + expense totals + current-month totals.
category-spending	Expense GROUP BY category + percentages.
monthly-trends	Monthly income and expense maps merged by year-month.
budget-utilization	BudgetService + expense spending.
goal-progress	FinancialGoalService + goal state.
financial-health	Income/expense + budgets + goals.
recent-activity	Recent expenses/incomes/contributions combined and sorted.

Monthly trends

```
incomeResults → monthlyIncome Map
expenseResults → monthlyExpense Map

key = year + "-" + month

months = union of keys

missing income/expense → BigDecimal.ZERO
final response → chronological sort
```

This is application-level orchestration: individual repositories know how to fetch data; DashboardService knows how to combine those data sets into product analytics.

Financial health

Component	Weight	Current rule
Savings	30	Positive savings rate scaled to 30; 20%+ full.
Expense control	25	≤50% full; 50–100% linearly decreases; ≥100% zero.
Budget	25	≤80% full; 80–120% decreases; ≥120% zero.
Goals	20	Average completion converted to 20 points.

This is deterministic business logic, not machine learning. The thresholds are product rules and should eventually be documented, tested and configurable if requirements demand it.

13. JPA/Hibernate — the database mental model

Technology	What it actually is
JPA	Specification/API for Java persistence.
Hibernate	Common implementation of JPA; performs ORM work.
Spring Data JPA	Repository abstraction and query derivation on top of JPA.
MySQL	Relational database storing actual rows.
JDBC driver	Allows Java database infrastructure to communicate with MySQL.

Entity example

```
@Entity
@Table(name = "expenses")
public class Expense {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;
}
```

@Entity tells JPA that the class is persistent. @Table maps it to a table. @Id identifies the primary key. IDENTITY delegates key generation to the database. @ManyToOne describes a many-expenses-to-one-user relationship. LAZY controls when the related object is loaded.

Repository derivation

```
findByIdAndUserId(id, userId)
findTop10ByUserIdOrderByExpenseDateDesc(userId)
findAllByUserId(userId)
```

Spring Data parses these method names and creates the implementation/query behavior. Learn the naming grammar: findBy, And, Or, OrderBy, Asc/Desc, Top/First, Between, GreaterThan, LessThan, etc.

14. Database integrity + financial correctness

Database concept	FinTrack relevance
Primary key	Every entity has a unique ID.
Foreign key	Expense/Budget → User/Category; Contribution → Goal.
UNIQUE	User email and Category name.
NOT NULL	Required entity fields.
DECIMAL(12,2)	Money storage mapping.
Cascade delete	Goal contribution rows can be removed with their goal in the intended schema.

Application checks and DB constraints are complementary. For example, `existsByEmail` improves the user-facing error, but a database `UNIQUE` constraint is still required because two concurrent requests could both pass the application check before either inserts.

Indexes

The uploaded source does not show an explicit index strategy for the most frequently filtered columns. As data grows, `user_id/date/category_id` combinations will become important candidates for indexes after measuring actual query plans.

PRODUCTION MINDSET

Do not add indexes blindly. Use query patterns, query plans and actual workload to decide which indexes provide value.

15. Testing — what exists vs what production needs

The uploaded project contains one test class with a `contextLoads()` test. That verifies the Spring application context can load; it does not verify business rules or endpoint behavior.

Test level	FinTrack tests to build
Unit	Health score, budget overlap, goal rules, date/amount validation.
Repository	Ownership queries, SUM, GROUP BY and filtering.
Controller	Request validation, status codes and JSON contract.
Security	No token, invalid token, valid token and wrong-user access.
Integration	Register → login → protected CRUD.
Concurrency	Concurrent goal contributions/updates.

POSTMAN VS AUTOMATED TESTS

Postman is excellent for manual/exploratory verification. Automated tests provide repeatable protection when code changes. We need both.

16. Exact source findings — prioritized engineering backlog

Priority	Finding	Why
P0	Hardcoded JWT secret + DB credentials in application.yml	Secret leakage / credential compromise.
P1	LoginRequest has no validation	Null/invalid input can reach authentication logic.
P1	Register password lacks @NotBlank	Null may bypass @Size.
P1	Login returns 200 with different failure messages	Wrong HTTP semantics + account enumeration.
P1	GlobalExceptionHandler misses InvalidExpenseException and CategoryNotFoundException	Inconsistent error contract.
P1	ValidationErrorResponse exists but validation handler is not shown	Validation errors may use framework default response.
P1	Goal contribution has no visible @Transactional	Two writes can become inconsistent.
P2	Budget status logic differs >= vs >	Exactly 100% can produce different statuses.
P2	Object[] aggregate projections	Runtime positional casting.
P2	Dashboard/budget mapping can trigger many aggregate queries	Potential performance scaling issue.
P2	DashboardRepository appears unused	Duplicate/ambiguous architecture.
P2	JwtUtil is empty	Dead code/no responsibility.
P3	Hello/Test controllers are development leftovers	Noise before production.
P3	Lombok dependency unused	Unnecessary dependency.
P3	ddl-auto=update/show-sql=true	Development settings need environment-specific production configuration.

These are not “your project is bad” findings. They are hardening topics that we should learn and fix deliberately.

17. Complete source inventory — nothing silently skipped

The uploaded project contains 78 application Java files. Below is the source inventory grouped by feature.

FINTRACKBACKENDAPPLICATION.JAVA

File	Type	Key methods
FintrackBackendApplication.java	FintrackBackendApplication	main

AUTH

File	Type	Key methods
auth/controller/AuthController.java	AuthController	register, login
auth/dto/LoginRequest.java	LoginRequest	getEmail, setEmail, getPassword, setPassword
auth/dto/RegisterRequest.java	RegisterRequest	getFirstName, setFirstName, getLastName, setLastName, getEmail, setEmail, getPassword, setPassword
auth/dto/response/LoginResponse.java	LoginResponse	getMessage, setMessage, getToken, setToken
auth/dto/response/RegisterResponse.java	RegisterResponse	getMessage, setMessage
auth/entity/User.java	User	getId, setId, getFirstName, setFirstName, getLastName, setLastName, getEmail, setEmail
auth/repository/UserRepository.java	UserRepository	—
auth/service/AuthService.java	AuthService	register, login

BUDGET

File	Type	Key methods
budget/controller/BudgetController.java	BudgetController	createBudget, getMyBudgets, getBudgetById, updateBudget, deleteBudget
budget/dto/BudgetPageResponse.java	BudgetPageResponse	getBudgets, setBudgets, getCurrentPage, setCurrentPage, getTotalPages, setTotalPages, getTotalElements, setTotalElements
budget/dto/BudgetResponse.java	BudgetResponse	getId, setId, getName, setName, getAmount, setAmount, getStartDate, setStartDate
budget/dto/CreateBudgetRequest.java	CreateBudgetRequest	getName, setName, getAmount, setAmount, getStartDate, setStartDate, getEndDate, setEndDate
budget/entity/Budget.java	Budget	getId, setId, getName, setName, getAmount, setAmount, getStartDate, setStartDate
budget/repository/BudgetRepository.java	BudgetRepository	—
budget/service/BudgetService.java	BudgetService	createBudget, filterBudgets, getBudgetById, updateBudget, deleteBudget, getAuthenticatedUser, getCategory, validateBudgetDates
budget/specification/BudgetSpecification.java	BudgetSpecification	filterBudgets

COMMON

File	Type	Key methods
common/exception/ApiResponse.java	ApiResponse	getStatus, setStatus, getMessage, setMessage
common/exception/BudgetNotFoundException.java	BudgetNotFoundException	—
common/exception/CategoryNotFoundException.java	CategoryNotFoundException	—
common/exception/EmailAlreadyExistsException.java	EmailAlreadyExistsException	—
common/exception/ExpenseNotFoundException.java	ExpenseNotFoundException	—
common/exception/FinancialGoalNotFoundException.java	FinancialGoalNotFoundException	—
common/exception/GlobalExceptionHandler.java	GlobalExceptionHandler	handleEmailAlreadyExists, handleExpenseNotFound, handleBudgetNotFound, handleInvalidBudget, handleIncomeNotFound, handleInvalidIncome, handleFinancialGoalNotFound, handleInvalidFinancialGoal
common/exception/IncomeNotFoundException.java	IncomeNotFoundException	—
common/exception/InvalidBudgetException.java	InvalidBudgetException	—
common/exception/InvalidExpenseException.java	InvalidExpenseException	—
common/exception/InvalidFinancialGoalException.java	InvalidFinancialGoalException	—
common/exception/InvalidIncomeException.java	InvalidIncomeException	—
common/exception/ValidationErrorResponse.java	ValidationErrorResponse	getStatus, setStatus, getMessage, setMessage, getErrors, setErrors

CONFIG

File	Type	Key methods
config/SecurityConfig.java	SecurityConfig	passwordEncoder, securityFilterChain

CONTROLLER

File	Type	Key methods
controller/HelloController.java	HelloController	sayHello
controller/TestController.java	TestController	home

DASHBOARD

File	Type	Key methods
dashboard/controller/DashboardController.java	DashboardController	getLoggedInUserId, getSummary, getCategoryWiseSpending, getMonthlyTrends, getBudgetUtilization, getGoalProgress, getFinancialHealth, getRecentActivity
dashboard/dto/BudgetUtilizationResponse.java	BudgetUtilizationResponse	getBudgetId, getBudgetName, getCategoryName, getBudgetAmount, getSpentAmount, getRemainingAmount, getPercentageUsed, getStatus
dashboard/dto/CategorySpendingResponse.java	CategorySpendingResponse	getCategoryId, getCategoryName, getTotalAmount, getPercentage, setCategoryId, setCategoryName, setTotalAmount, setPercentage

File	Type	Key methods
dashboard/dto/DashboardSummaryResponse.java	DashboardSummaryResponse	getTotalIncome, getTotalExpense, getTotalSavings, getSavingsRate, getCurrentMonthIncome, getCurrentMonthExpense, setTotalIncome, setTotalExpense
dashboard/dto/FinancialHealthResponse.java	FinancialHealthResponse	getScore, getRating, getSavingsScore, getExpenseControlScore, getBudgetScore, getGoalScore
dashboard/dto/GoalProgressResponse.java	GoalProgressResponse	getGoalId, getGoalName, getTargetAmount, getCurrentAmount, getRemainingAmount, getPercentageCompleted, getTargetDate, getStatus
dashboard/dto/MonthlyTrendResponse.java	MonthlyTrendResponse	getYear, getMonth, getMonthName, getTotalIncome, getTotalExpense, getTotalSavings
dashboard/dto/RecentActivityResponse.java	RecentActivityResponse	getType, getTitle, getAmount, getDate
dashboard/repository/DashboardRepository.java	DashboardRepository	—
dashboard/service/DashboardService.java	DashboardService	getSummary, getCategoryWiseSpending, getMonthlyTrends, getFinancialHealth, getRecentActivity

EXPENSE

File	Type	Key methods
expense/controller/ExpenseController.java	ExpenseController	createExpense, getMyExpenses, getExpenseById, updateExpense, deleteExpense
expense/dto/CreateExpenseRequest.java	CreateExpenseRequest	getTitle, setTitle, getAmount, setAmount, getDescription, setDescription, getExpenseDate, setExpenseDate
expense/dto/ExpensePageResponse.java	ExpensePageResponse	getExpenses, setExpenses, getCurrentPage, setCurrentPage, getTotalPages, setTotalPages, getTotalElements, setTotalElements
expense/dto/ExpenseResponse.java	ExpenseResponse	getId, setId, getTitle, setTitle, getAmount, setAmount, getDescription, setDescription
expense/entity/Category.java	Category	getId, setId, getName, setName
expense/entity/Expense.java	Expense	getId, setId, getTitle, setTitle, getAmount, setAmount, getDescription, setDescription
expense/repository/CategoryRepository.java	CategoryRepository	—
expense/repository/ExpenseRepository.java	ExpenseRepository	—
expense/service/ExpenseService.java	ExpenseService	createExpense, filterExpenses, getExpenseById, updateExpense, deleteExpense, getAuthenticatedUser, getCategory, validateExpenseDate
expense/specification/ExpenseSpecification.java	ExpenseSpecification	filterExpenses

GOAL

File	Type	Key methods
goal/controller/FinancialGoalController.java	FinancialGoalController	createGoal, getMyGoals, getGoalById, updateGoal, deleteGoal, addContribution, getContributions
goal/dto/CreateContributionRequest.java	CreateContributionRequest	getAmount, getContributionDate, getNote, setAmount, setContributionDate, setNote
goal/dto/CreateFinancialGoalRequest.java	CreateFinancialGoalRequest	getName, getTargetAmount, getTargetDate, getDescription, setName, setTargetAmount, setTargetDate, setDescription
goal/dto/FinancialGoalResponse.java	FinancialGoalResponse	getId, getName, getTargetAmount, getCurrentAmount, getRemainingAmount, getPercentageCompleted, getTargetDate, getDescription
goal/dto/GoalContributionPageResponse.java	GoalContributionPageResponse	getContributions, getCurrentPage, getTotalPages, getTotalElements, getPageSize, setContributions, setCurrentPage, setTotalPages
goal/dto/GoalContributionResponse.java	GoalContributionResponse	getId, getAmount, getContributionDate, getNote, setId, setAmount, setContributionDate, setNote
goal/dto/UpdateFinancialGoalRequest.java	UpdateFinancialGoalRequest	getName, getTargetAmount, getTargetDate, getDescription, setName, setTargetAmount, setTargetDate, setDescription
goal/entity/FinancialGoal.java	FinancialGoal	getId, getName, getTargetAmount, getCurrentAmount, getTargetDate, getDescription, getUser, getCreatedAt
goal/entity/GoalContribution.java	GoalContribution	getId, getAmount, getContributionDate, getNote, getGoal, getCreatedAt, setId, setAmount
goal/repository/FinancialGoalRepository.java	FinancialGoalRepository	—
goal/repository/GoalContributionRepository.java	GoalContributionRepository	—
goal/service/FinancialGoalService.java	FinancialGoalService	createGoal, getMyGoals, getGoalById, updateGoal, deleteGoal, addContribution, getOwnedGoal, getAuthenticatedUser
goal/specification/GoalContributionSpecification.java	GoalContributionSpecification	filterContributions

INCOME

File	Type	Key methods
income/controller/IncomeController.java	IncomeController	createIncome, getMyIncomes, getIncomeById, updateIncome, deleteIncome
income/dto/CreateIncomeRequest.java	CreateIncomeRequest	getSource, getAmount, getDescription, getIncomeDate, getIncomeType, setSource, setAmount, setDescription
income/dto/IncomePageResponse.java	IncomePageResponse	getIncomes, getCurrentPage, getTotalPages, getTotalElements, getPageSize, setIncomes, setCurrentPage, setTotalPages
income/dto/IncomeResponse.java	IncomeResponse	getId, getSource, getAmount, getDescription, getIncomeDate, getIncomeType, setId, setSource
income/entity/Income.java	Income	getId, getSource, getAmount, getDescription, getIncomeDate, getIncomeType, getUser, getCreatedAt
income/repository/IncomeRepository.java	IncomeRepository	—

File	Type	Key methods
income/service/IncomeService.java	IncomeService	createIncome, filterIncomes, getIncomeById, updateIncome, deleteIncome, getAuthenticatedUser, validateIncomeDate, validatePagination
income/specification/IncomeSpecification.java	IncomeSpecification	filterIncomes

SECURITY

File	Type	Key methods
security/jwt/JwtAuthenticationFilter.java	JwtAuthenticationFilter	doFilterInternal
security/jwt/JwtService.java	JwtService	getSigningKey, generateToken, extractEmail, isTokenValid
security/jwt/JwtUtil.java	JwtUtil	—

SERVICE

File	Type	Key methods
service/HelloService.java	HelloService	sayHello

18. Our new learning protocol from this point onward

NO MORE BLIND COPY-PASTE

For every new feature: **Requirement** → **architecture** → **technical theory** → **data model** → **API contract** → **implementation** → **important lines explained** → **testing** → **failure/security/concurrency cases** → **interview explanation** → **Git commit**.

Level	What you should eventually be able to say
Simple	"This class receives the request and passes it to the service."
Technical	"Spring maps the request to a DTO, validates it, injects the service bean, and the service uses a repository backed by Spring Data JPA."
Professional	"The controller is the HTTP boundary, the service owns the use case/business invariants, the repository owns persistence access, and ownership is enforced through user-scoped queries."

For every code block I give you, I will also teach the relevant Java/Spring/JPA/security/database concept. You should understand the important lines before we move on.

Mastery checklist

Concept	Status
Controller → Service → Repository → DB flow	■ Explain
Spring Bean + Dependency Injection	■ Explain
DTO + Jackson + validation	■ Explain
JPA vs Hibernate vs Spring Data JPA	■ Explain
Entity mapping and relationships	■ Explain
Derived repository queries	■ Explain
JPQL + @Query + @Param	■ Explain
Specification + Criteria API	■ Explain
Page/Pageable/Sort	■ Explain
BCrypt	■ Explain
JWT + filter chain + SecurityContext	■ Explain
Authentication vs authorization	■ Explain
Ownership/data isolation	■ Explain
Transactions	■ Explain
Concurrency/race conditions	■ Explain
Testing strategy	■ Explain
Production configuration/secrets	■ Explain

19. Final professional assessment

After inspecting the actual ZIP, the important conclusion is this: you have already built a meaningful Spring Boot backend. The missing piece is not more CRUD. The missing piece is the engineering understanding underneath the CRUD.

THE TARGET

You should be able to open any FinTrack backend file and explain: **what it does, why it exists, how Spring uses it, how the request reaches it, how it interacts with JPA/MySQL, what can fail, what security rule it enforces, and how you would improve it for production.**

Next study order: Spring IoC/DI → Spring MVC request lifecycle → DTO/validation → JPA/Hibernate → repositories/JPQL → Specifications → transactions/concurrency → Security/JWT → automated testing → production hardening.

I also attempted to run the project's Maven tests from the uploaded source, but the Maven wrapper could not download its Maven distribution because this execution environment has no usable network access. Therefore this audit does not claim a fresh compile/test pass from the ZIP.